

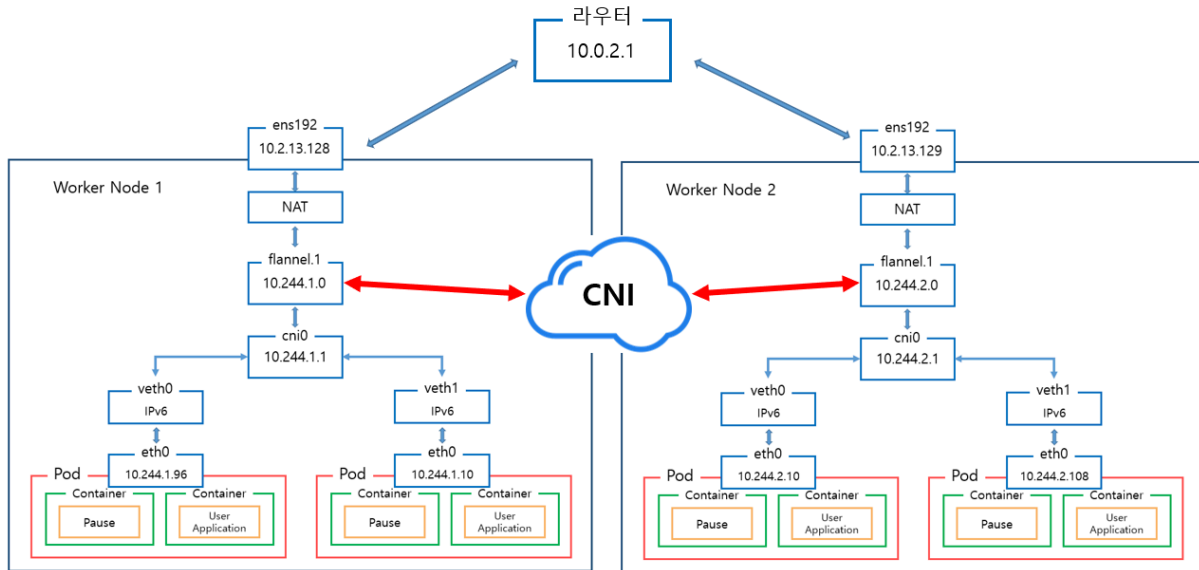
Primary CNI - week 4

항목	Flannel	Calico	Cilium
기본 철학	단순한 Pod Connectivity	L3 Routing + Security	eBPF 기반 Networking + Security
대표 Datapath	Linux Bridge + VXLAN	Linux Routing	eBPF + Linux Networking
Overlay	VXLAN	VXLAN / IPIP	VXLAN / Geneve
Native Routing	제한적/host-gw	강력한 지원	지원
BGP	X	지원	지원 가능
NetworkPolicy	flanneld 자체 핵심 기능 아님	강력하게 지원	강력하게 지원
Policy 구현	별도 정책 구성 필요	iptables/nftables/eBPF 등	eBPF
IPAM	Node subnet 중심	Calico IPAM	Cilium IPAM
구조	단순	중간~복잡	상대적으로 복잡
주요 특징	간단한 Overlay	Routing/BGP	eBPF/Identity

Flannel Architecture

Kubernetes 클러스터에서 Pod 간의 네트워크 통신을 가능하게 해주는 CNI 플러그인

- **오버레이 네트워크(Overlay Network):** 기본적으로 **VXLAN** 방식을 사용하여 파드 간의 원래 패킷을 UDP 패킷으로 한 번 더 캡슐화하여 노드 간 통신을 수행 (MTU 감소, 성능 오버헤드)



• 구성 요소

구성 요소	역할
<code>flanneld</code>	각 노드에서 실행되는 에이전트. 다른 노드의 네트워크 정보를 확인하고 경로·VXLAN 장치 등을 설정
Flannel CNI plugin	Pod 네트워크 생성 시 Flannel의 서브넷·MTU 정보를 읽어 다른 CNI 플러그인에 작업을 위임
<code>host-local</code> IPAM	해당 노드의 Pod 대역에서 개별 Pod IP를 할당
<code>cni0</code>	일반적인 구성에서 노드 내부 Pod들이 연결되는 Linux bridge
<code>flannel.1</code>	VNI가 1인 일반적인 구성의 VXLAN 인터페이스. 터널 종단점 역할

flanneld

각 노드에서 실행되면서, 다른 노드의 Pod로 패킷을 보낼 수 있도록 Linux 네트워크를 설정하고 관리하는 데몬

Kubernetes API를 통해 각 Node의 Pod 대역과 터널 정보를 조회·감시하고, 이를 바탕으로 자기 노드의 네트워크를 설정

• flanneld가 관리하는 정보

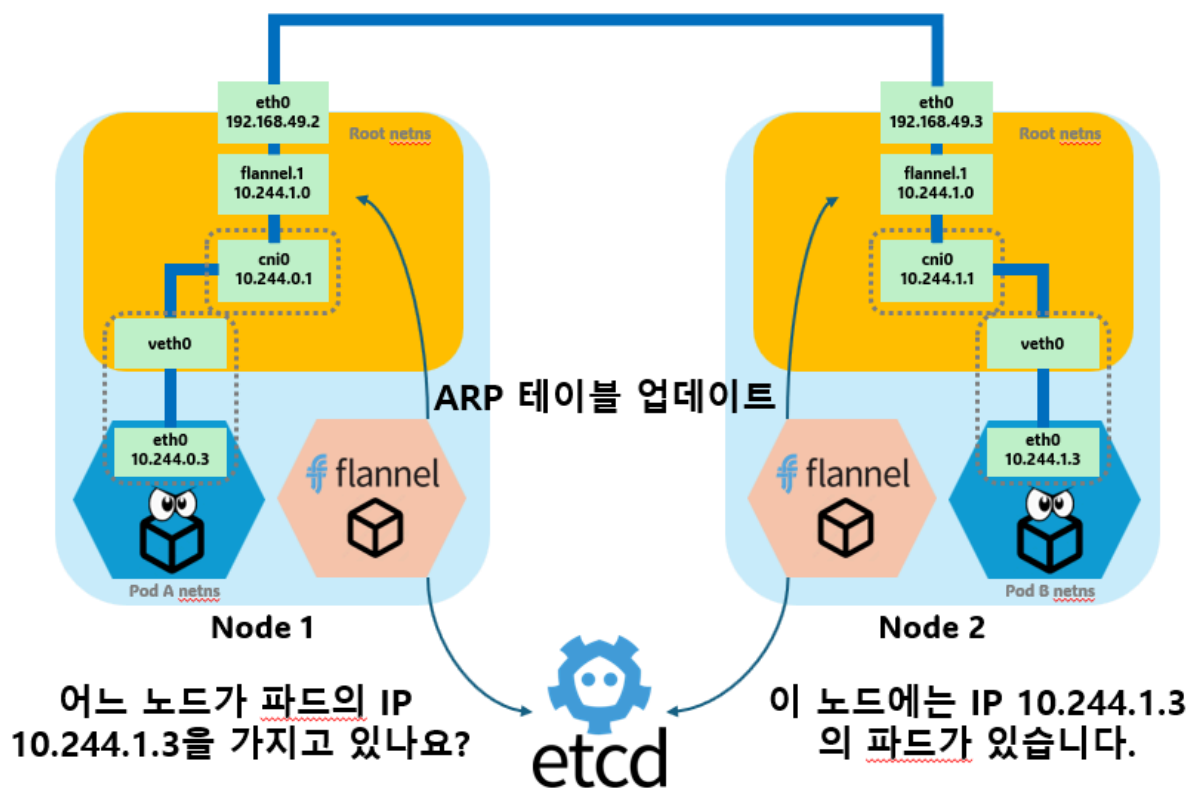
정보	예시	의미
전체 Pod 네트워크	<code>10.244.0.0/16</code>	클러스터에서 사용하는 Pod 주소 범위
자기 노드의 Pod 대역	<code>10.244.1.0/24</code>	이 노드의 Pod에 사용할 주소 범위
노드 간 통신용 IP	<code>10.2.13.128</code>	다른 노드가 이 노드에 접근하는 주소
백엔드	<code>vxlan</code>	노드 간 트래픽을 전달하는 방식

정보	예시	의미
원격 노드 정보	Pod 대역, 노드 IP, VTEP MAC	원격 Pod로 가는 경로를 설정하는 재료

flannel.1 (UDP 8472)

다른 노드에 위치한 파드 간 통신을 가능하게 해주는 VXLAN VTEP (터널 종단점) 역할

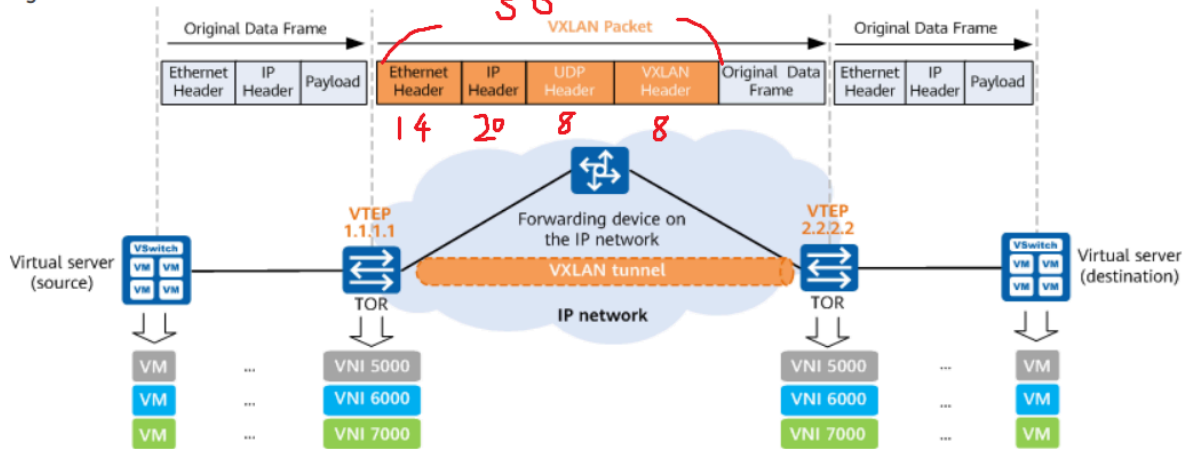
- 송신 시: 내부 Ethernet 프레임을 VXLAN·UDP·IP로 감싸서 전송
- 수신 시: 바깥 헤더를 제거해 내부 Ethernet 프레임을 꺼냄



VXLAN

Ethernet 프레임을 UDP/IP 패킷 안에 넣어서, IP 네트워크를 통해 전달하는 기술

Figure 1-6 VXLAN network model



Flannel 장단점

구분	항목	설명
장점	단순한 구성	Pod 간 통신에 집중해 설치와 구조 이해가 비교적 쉬움
장점	기반 네트워크 변경 최소화	기존 라우터에 각 Pod 대역의 경로를 등록할 필요가 없음
장점	노드 간 네트워크 유연성	노드 간 IP·UDP 통신이 가능하면 서로 다른 서브넷도 연결 가능
장점	커널 기반 처리	Linux 커널이 VXLAN 캡슐화와 패킷 전달을 수행
단점	캡슐화 오버헤드	추가 헤더와 처리 비용이 발생하고, Pod가 사용할 MTU가 줄어듦
단점	NetworkPolicy 별도 구성	기본 Flannel만으로는 정책을 집행하지 않아 별도 구성 요소가 필요
단점	기본 VXLAN은 암호화 없음	트래픽 암호화가 필요하면 WireGuard 백엔드 등 추가 검토 필요
단점	장애 분석 복잡성	Pod 경로뿐 아니라 VXLAN, MTU, 노드 간 네트워크도 확인해야 함

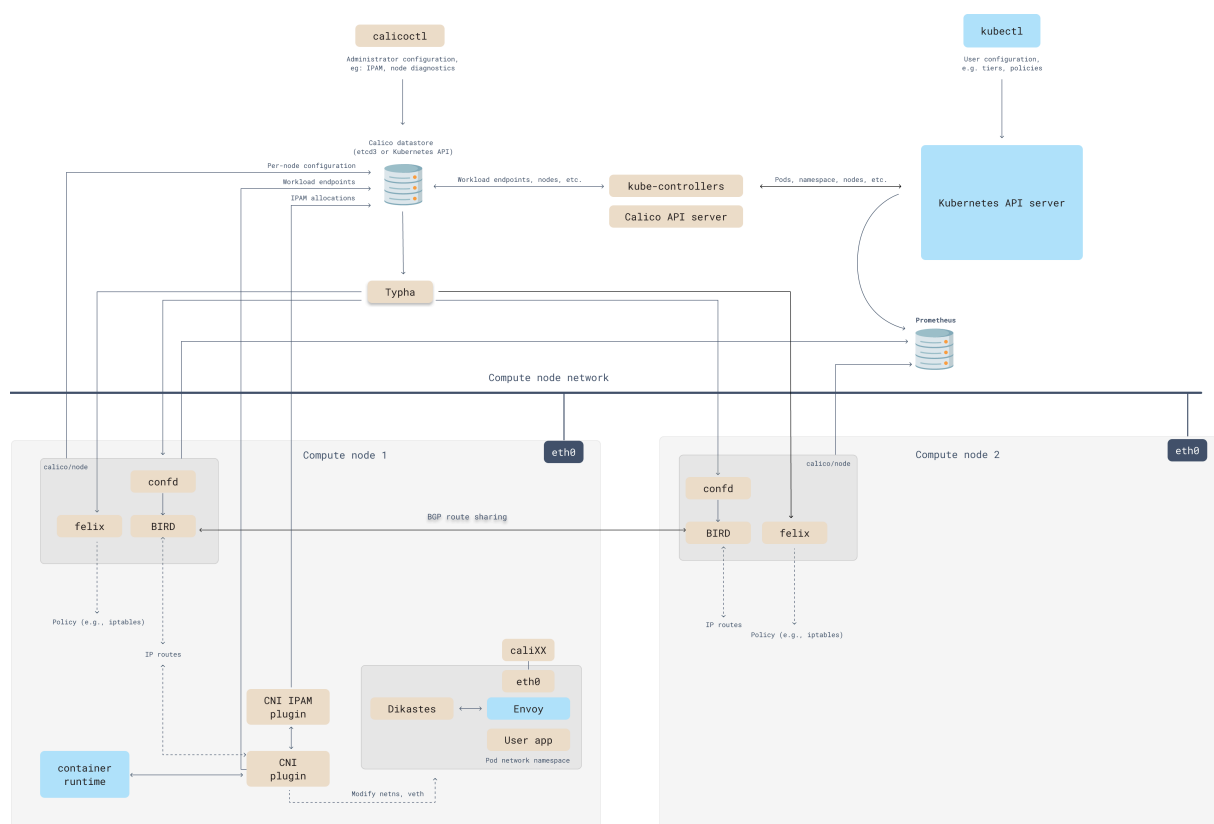
Flannel Backend (노드 간 패킷 전달 방식)

백엔드	전달 방식	특징·사용 조건
VXLAN	Ethernet 프레임을 UDP/IP로 캡슐화	일반적인 권장 선택. 노드가 서로 다른 서브넷에 있어도 사용 가능
host-gw	원격 노드 IP를 다음 홉으로 지정해 직접 라우팅	캡슐화 오버헤드가 없음. 노드 간 직접적인 L2 연결 필요

백엔드	전달 방식	특징·사용 조건
WireGuard	WireGuard 터널로 캡슐화·암호화	노드 간 트래픽 암호화가 필요할 때 사용
UDP	별도의 UDP 백엔드로 캡슐화	VXLAN과는 다른 방식. 공식 문서에서는 디버깅·구형 환경 용도로 제한적으로 권장

Calico Architecture

컨테이너, 가상 머신 및 기본 호스트 기반 워크로드를 위한 오픈 소스 네트워킹 및 네트워크 보안 솔루션



구성요소

- Felix (펠릭스) : 인터페이스 관리, 라우팅 정보 관리, ACL 관리, 상태 체크
- BIRD (버드): BGP Peer 에 라우팅 정보 전파 및 수신, BGP RR(Route Reflector)
- Confd : calico global 설정과 BGP 설정 변경 시(트리거) BIRD 에 적용해줌

- Datastore plugin : calico 설정 정보를 저장하는 곳 - k8s API datastore(kdd) 혹은 etcd 중 선택
- Calico IPAM plugin : 클러스터 내에서 파드에 할당할 IP 대역
- calico-kube-controllers : calico 동작 관련 감시(watch)
- calicoctl : calico 오브젝트를 CRUD 할 수 있다, 즉 datastore 접근 가능

Calico 특징

특징	설명
L3 라우팅 중심	Pod IP를 목적지로 호스트가 패킷을 라우팅
네트워크 정책 지원	Pod·Namespace의 label, IP, 포트 등을 기준으로 통신 제어
다양한 전달 방식	비캡슐화 라우팅, IP-in-IP, VXLAN 지원
BGP 연동	노드나 외부 라우터와 Pod 네트워크 경로를 교환할 수 있음
IP 주소 관리	Calico IPAM으로 Pod IP와 주소 블록 관리
eBPF 데이터 플레인 선택 가능	eBPF로 패킷·정책을 처리하고 Kubernetes Service 네트워킹도 구현 가능

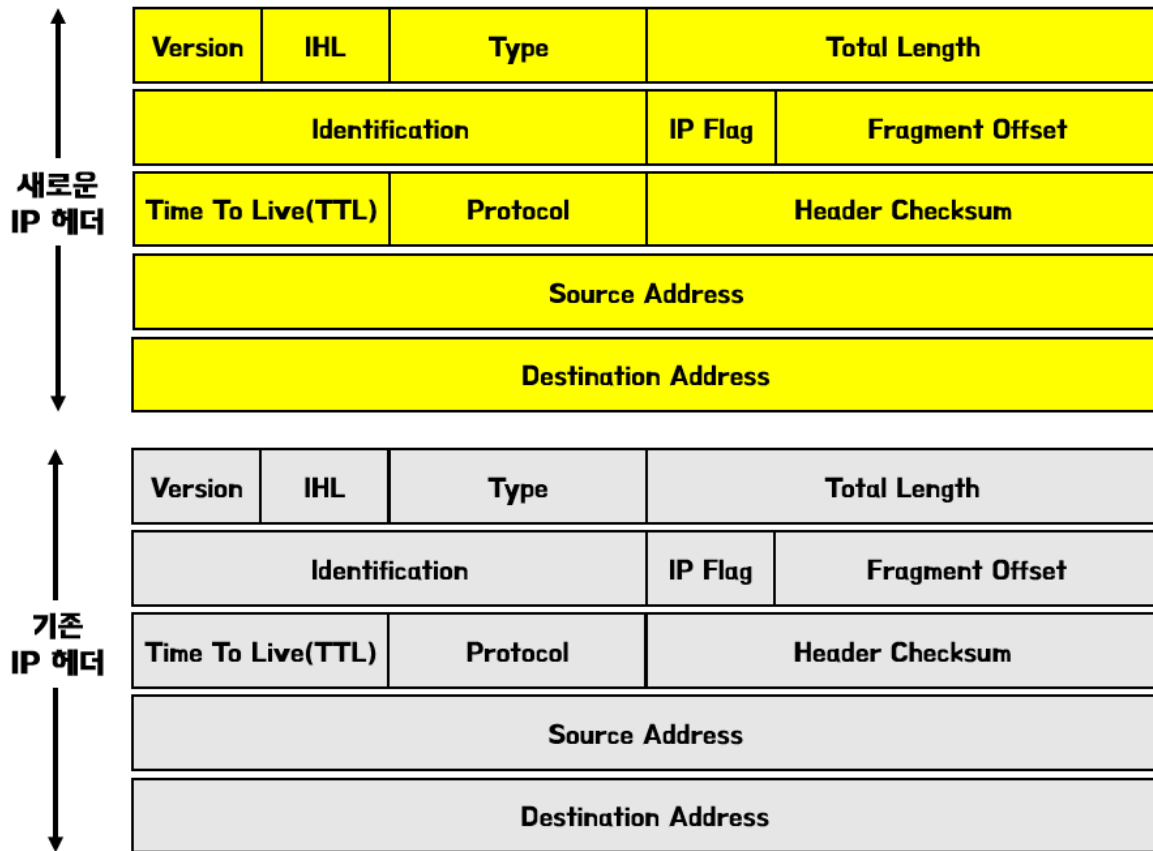
- **BGP 프로토콜**: 스위치나 라우터 등의 네트워크 장비에서 통신하기 위한 상대의 ip 대역을 전파하기 위해 사용되는 프로토콜

패킷 전달 방식

전달 구성	동작	고려 사항
Direct (비캡슐화 라우팅)	원래 Pod IP 패킷을 그대로 전달	기반 네트워크가 Pod 트래픽을 올바르게 라우팅해야 함
IP-in-IP	Pod IP 패킷 바깥에 IP 헤더 추가	Calico에서는 IPv4 대상
VXLAN	내부 Ethernet 프레임에 UDP/IP로 감쌈	VXLAN 풀만 사용하고 외부 BGP 경로 광고가 필요 없다면 BGP 없이 구성 가능

IP-in-IP (IPIP 프로토콜)

IP 패킷 바깥에 IP 헤더를 하나 더 붙여서 전달하는 터널링 방식



사용 이유: Pod의 IP 대역을 모르는 기반 네트워크를 통과하기 위해 사용 (ex. 다른 서브넷)

- 프로토콜

프로토콜 값	이름	의미
01	ICMP	다음 데이터가 ICMP 헤더임
06	TCP	다음 데이터가 TCP 헤더임
11(17)	UDP	다음 데이터가 UDP 헤더임
04	IPv4(IPIP)	다음 데이터가 또 다른 IPv4 헤더임 (IPIP)

IP-in-IP vs VXLAN

항목	IP-in-IP	VXLAN
캡슐화 대상	IP 패킷(L3)	Ethernet 프레임(L2)
터널 운반 방식	IP 안에 IP	UDP/IP 안에 Ethernet
외부 IP의 Protocol 값	4: IP-in-IP	17: UDP
터널용 포트	없음	UDP 목적지 포트 사용
네트워크 식별자	VNI 없음	24비트 VNI

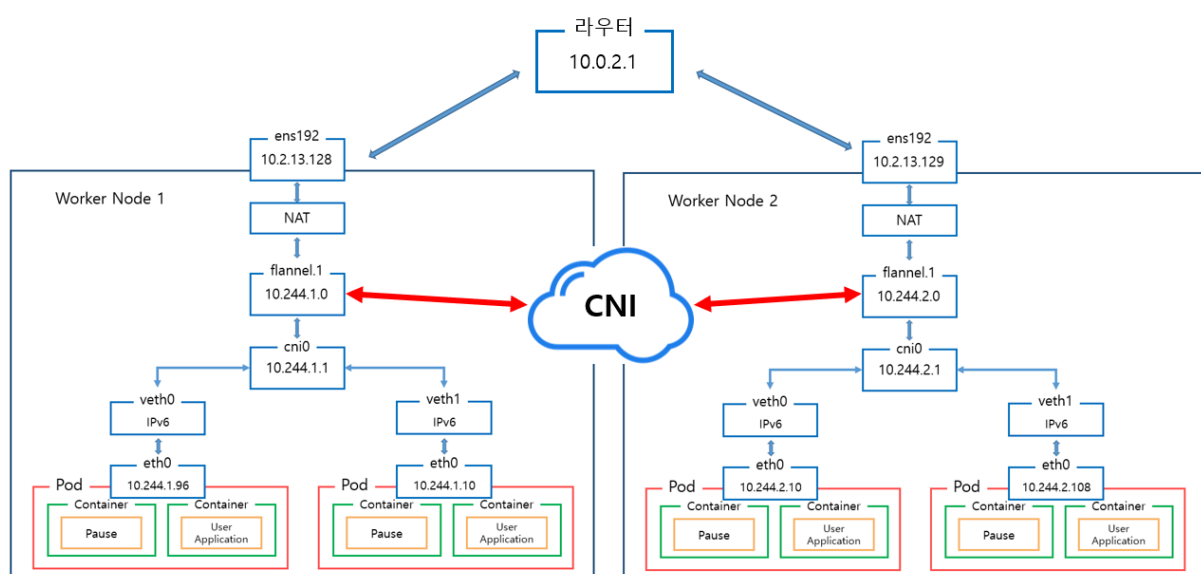
항목	IP-in-IP	VXLAN
IPv4 기반 오버헤드 ¹	20바이트	50바이트
경로 MTU 1500일 때 내부 IP 최대 ¹	1480바이트	1450바이트
자체 암호화	없음	없음

사용 판단 정리

판단 기준	IP-in-IP	VXLAN
헤더·MTU 효율	추가 헤더가 작아 내부 패킷 공간이 더 큼	추가 헤더가 더 큼
기반 네트워크 조건	IP 프로토콜 4 허용 필요	설정된 UDP 포트 허용 필요
Calico 경로 구성	앞서 본 구성에서는 BGP와 함께 사용	BGP 없이 구성 가능
IP 버전	Calico에서는 IPv4	Calico에서 IPv4·IPv6 지원 조건 확인
운영 시 확인 사항	BGP 경로, IPIP 허용 여부, MTU	VXLAN 전달 정보, UDP 허용 여부, MTU

CNI 별 Packet flow 비교

Flannel VXLAN



1. Pod에서 패킷 생성 (10.244.1.96 → 10.244.2.10)

2. 호스트로 전달

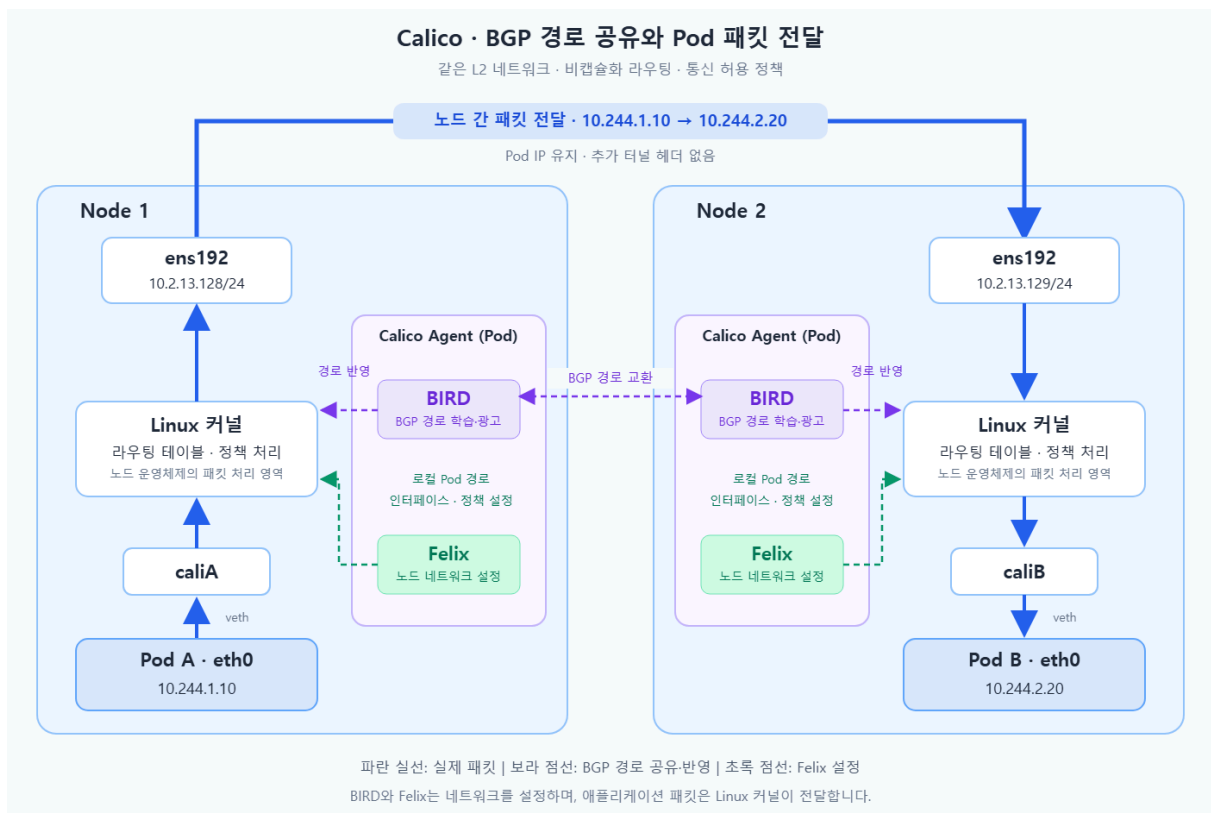
Pod의 `eth0 → veth0 → cni0` 를 거쳐 호스트의 라우팅 경로로 들어갑니다.

3. flannel.1 에서 VXLAN 캡슐화 (외부 IP를 10.2.13.128 → 10.2.13.129 로 설정)

4. 노드 간 전달

5. 역캡슐화 후 목적지 Pod로 전달

Calico Direct (비캡슐화)



0. BIRD가 BGP로 원격 Pod 경로를 공유해 커널에 반영하고, Felix가 로컬 Pod 경로와 정책을 설정

1. Pod에서 패킷 생성 (10.244.1.10 → 10.244.2.20)

2. 호스트로 전달

Pod A의 `eth0` 에서 veth 쌍으로 연결된 호스트 인터페이스 `caliA` 로 전달합니다.

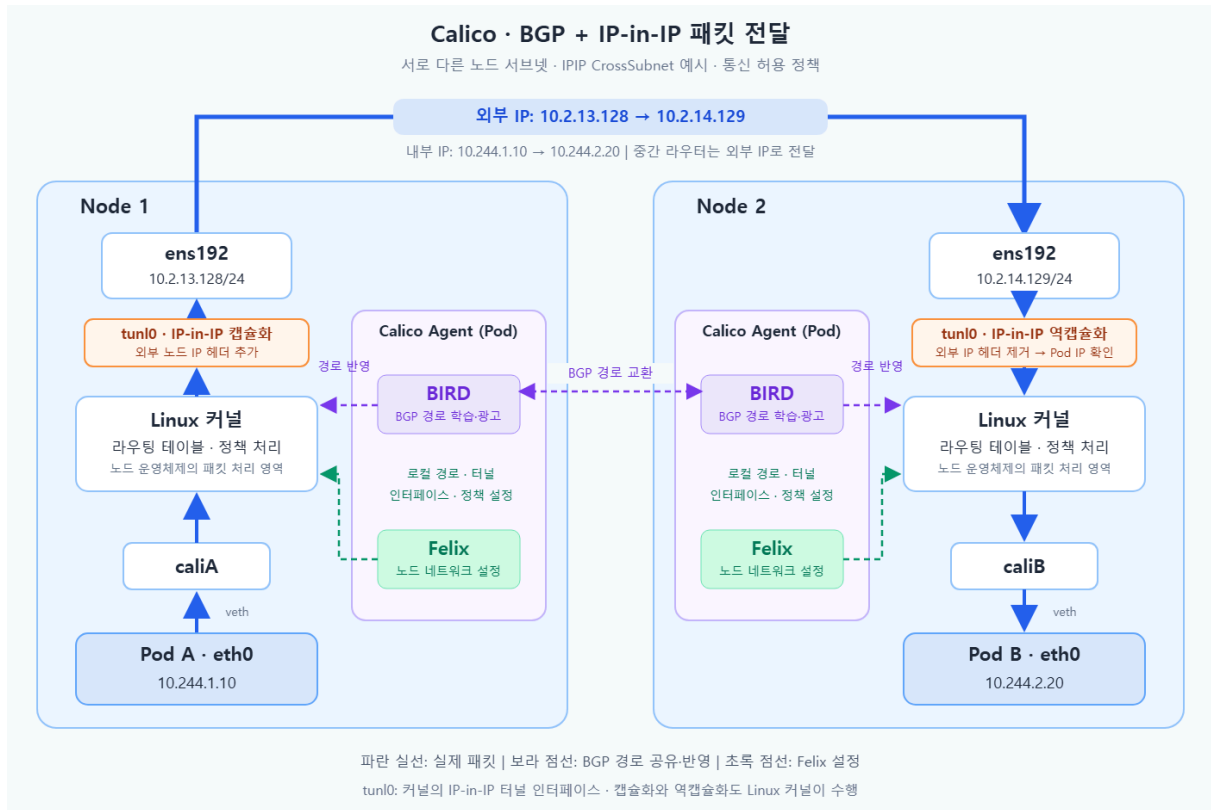
3. Node 1에서 정책 검사·라우팅

커널이 통신 허용 여부를 확인 및 Node 2의 `10.2.13.129` 를 다음 홉으로 선택

4. 캡슐화 없이 노드 간 전달 (Pod 출발지·목적지 IP를 유지하며, 터널 헤더를 추가 X)

5. Node 2에서 목적지 Pod로 전달

Calico IP-in-IP



1. BIRD가 BGP로 원격 Pod 경로를 공유해 커널에 반영하고, Felix가 로컬 Pod 경로·터널·정책을 설정

2. Pod에서 패킷 생성 (10.244.1.10 → 10.244.2.20)

3. 호스트로 전달

Pod A의 eth0 에서 veth 쌍으로 연결된 호스트 인터페이스 caliA 로 전달합니다.

4. Node 1에서 정책 검사·라우팅

5. IP-in-IP 캡슐화 후 노드 간 전달

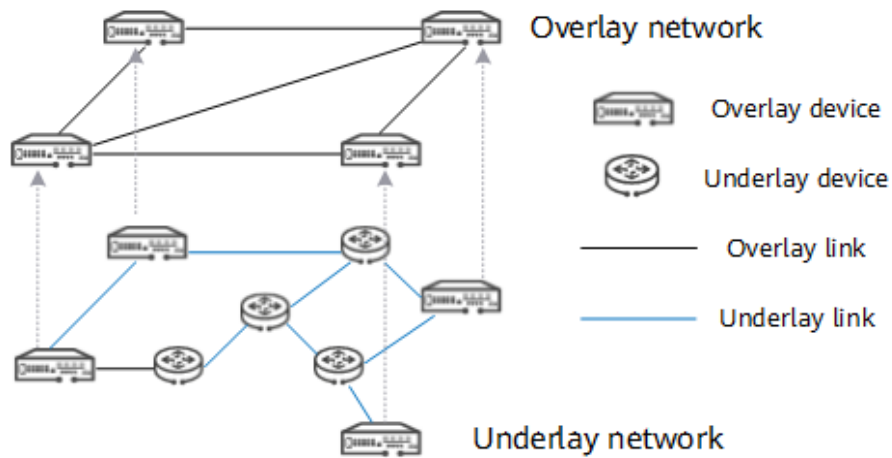
- 내부 Pod IP를 유지한 채 외부 IP 헤더(10.2.13.128 → 10.2.14.129)를 추가
- 중간 라우터는 외부 노드 IP를 기준으로 전달

6. Node 2에서 역캡슐화

ens192 로 받은 패킷을 커널의 IP-in-IP 처리로 역캡슐화하여, 외부 IP 헤더를 제거하고 내부 목적지 10.244.2.20 을 확인

7. Node 2에서 목적지 Pod로 전달

Overlay vs Underlay



항목	언더레이 네트워크	오버레이 네트워크
개념	실제 통신 경로를 제공하는 기반 네트워크	기반 네트워크 위에 구성한 논리적 네트워크
주요 역할	장치·터널 종단점 사이에서 패킷 운반	실제 위치와 별개로 논리적인 연결·분리 제공
구성 요소	NIC, 스위치, 라우터, 링크, IP 경로	가상 인터페이스, 터널 종단점, 논리적 경로
예시	집 LAN, 통신사 망, 인터넷 IP 네트워크	VPN, VXLAN 네트워크, P2P 네트워크
데이터 전송	기반 링크와 라우팅 경로를 따라 전달	논리적 연결을 사용하되, 실제 운반은 언더레이에 의존
주소 — 터널 기준	외부 IP로 터널 종단점까지 전달	내부 IP로 실제 통신 대상 식별
캡슐화·오버헤드	기반 네트워크의 Ethernet/IP 헤더 사용	터널 헤더 추가에 따른 처리 비용과 패킷 크기 증가
MTU	경로에서 전달 가능한 패킷 크기를 제한	언더레이 경로 MTU에서 터널 오버헤드를 고려해야 함
패킷 처리 구현	하드웨어·소프트웨어 모두 가능	하드웨어·소프트웨어 모두 가능
배포·변경	장비·배선 변경이 필요하면 시간이 소요되며 자동화도 가능	기반망 변경을 줄이고 논리적 네트워크를 추가·변경 가능
ECMP	동일 비용의 여러 경로로 트래픽 분산 가능	터널 트래픽이 언더레이의 ECMP를 활용 가능

항목	언더레이 네트워크	오버레이 네트워크
확장성	토폴로지·장비 용량·라우팅 설계에 따라 결정	논리적 확장이 유연하지만 기반망 용량과 상태 관리에 제약
멀티테넌트·격리	VLAN·VRF 등으로 분리 가능	VNI 등 식별자, 전달 테이블, 정책으로 분리 가능
관련 기술	Ethernet, IP, OSPF, BGP 등	캡슐화: VXLAN·GRE·IP-in-IP 등 / 제어 평면: BGP EVPN 등
장애 분석	링크, IP 도달성, 라우팅, MTU 확인	언더레이 상태에 더해 터널·논리적 경로·정책 확인
의존 관계	오버레이의 내부 구조를 몰라도 패킷 운반 가능	언더레이의 연결성과 전송 품질에 의존

언더레이·오버레이는 상대적 → 인터넷은 VPN 입장에서 언더레이

▼ 참고자료

https://hhhyunwoo.github.io/posts/eng_k8s-flannel-networking/

<https://themapisto.tistory.com/267>

<https://holy-hoon.tistory.com/2>

<https://white-polarbear.tistory.com/69>